

No. Questions

What is Docker?

Docker Installation?

Why Docker is Used?

Advantages of Docker?

TO check version?

Check the all docker Commands

Difference Between Docker and Virtual Machine?

No. Docker images

list all images?

Download/Pull image from Docker Hub

Delete Docker Image

No. Docker container

Show running containers?

Show all running containers?

Delete Docker Image

Docker Basic

What is Docker

- Docker is an open-source platform that allows developers to package, deploy, and run applications as containers. A container is a lightweight, portable environment that houses an application and all of its dependencies.
- The application can be run anywhere (Windows, Linux, Cloud) in the same way.
- We can run code and dependencies in containers without installing them directly on your system.
-
- Docker is an **open-source containerization platform** that allows developers to package, deploy, and run applications inside containers.
- A container is a lightweight, portable, and isolated(Isolated ka matlab hota hai alag ya separate environment me chalna) environment that contains an application along with all its required dependencies, libraries, and configurations.
- Docker ensures that the application runs consistently across different environments such as Windows, Linux, and cloud platforms.
- It allows us to run applications and their dependencies inside containers without installing those dependencies directly on the host operating system.

Docker Installation

- search **get docker** on google (OR) hit "<https://docs.docker.com/get-started/get-docker/>"
- click on Docker desktop for window then click on **Docker Desktop for Windows - x86_64**

Why Docker is Used?

1. Consistent Environment
2. Dependency Management
3. Easy Deployment
4. Faster Startup
5. Scalability
6. Isolation
7. CI/CD Integration

Advantages of Docker

1. Lightweight
2. Fast Startup
3. Portability
4. Consistency
5. Efficient Resource Utilization
6. Easy Scaling
7. Isolation
8. Simplified Dependency Management
9. Better CI/CD

10. Microservices Friendly

TO check version

- Shows installed Docker version

```
docker --version  
docker -v
```

- Shows client + server version

```
docker version
```

- Shows full system info (images, containers, CPU, storage, etc.)

```
docker info
```

Check the all docker Commands

```
# opne cmd  
docker
```

Difference Between Docker and Virtual Machine

- Docker host OS ka kernel(Kernel Operating System ka core (heart) hota hai jo hardware aur software ke beech communication karwata hai.) share karta hai, jabki VM har instance ke liye alag Guest OS chalata hai.
- Docker lightweight hota hai, jabki VM heavy hoti hai.
- Docker seconds me start ho jata hai, VM ko start hone me zyada time lagta hai.
- Docker kam RAM aur CPU use karta hai, VM zyada resources consume karti hai.
- Docker me containers run hote hain, VM me virtual machines run hoti hain.
- Docker ki performance generally better hoti hai kyunki full OS load nahi hota.
- VM stronger isolation provide karti hai kyunki har VM ka apna OS hota hai.
- Docker applications ko package aur deploy karne ke liye use hota hai, VM poora operating system virtualize karti hai.

Docker images

List all local images

```
docker images
# (OR)
docker image ls
```

Output:-

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
myapp	latest	a1b2c3d4e5f6	2 minutes ago	180MB
nginx	latest	f7g8h9i0j1k2	3 days ago	192MB
python	3.12	l3m4n5o6p7q8	1 week ago	1.02GB

Download/Pull image from Docker Hub

```
docker pull <image_name>
# (OR)
docker image pull <image_name>
```

```
docker pull nginx
docker pull hello-world
docker pull mysql
```

Download a specific version

```
docker pull python:3.12
docker pull mysql:8.0
```

Delete Docker Image

- Docker image ko delete karne ke liye `docker rmi <image_name>` ya `docker image rm <image_name>` command use karte hain. Agar image kisi container se linked ho to `-f` flag se force delete kar sakte hain.
- By Image name/ image id

```
docker image rmi/rm image_name/image_id
```

- **Force Delete:-** Agar image kisi container dwara use ho rahi ho:

```
docker image rmi -f nginx
# OR
```

```
docker image rm -f nginx
```

- remove unused images remove

```
docker image prune
```

What Are Docker Image Layers?

- A Docker Image is made up of multiple read-only layers stacked on top of each other.
- Each instruction in a Dockerfile creates a new layer.
- **HINDI:-**
- Docker Image एक single file नहीं होती, बल्कि कई layers (परतों) से मिलकर बनी होती है।
- Dockerfile में लिखी गई लगभग हर instruction एक नई layer बनाती है।

Example

```
FROM ubuntu:22.04
RUN apt update
RUN apt install -y nginx
COPY . /app
CMD ["nginx", "-g", "daemon off;"]

#
Layer 1: Ubuntu Base Image
Layer 2: apt update
Layer 3: nginx installation
Layer 4: Copy application files
Layer 5: CMD instruction
```

Why Layers are Useful?

1. Fast Build (Caching)
- अगर आपने सिर्फ code बदला:

```
COPY . /app
```

- तो Docker केवल इस layer को rebuild करेगा।
 - apt install वाली layer दोबारा नहीं चलेगी।
 - इससे build बहुत तेज हो जाता है।
2. Storage Save होता है
- मान लो दो images हैं:

- Image A = Ubuntu + Python
- Image B = Ubuntu + Nginx
- दोनों Ubuntu वाली layer को share करेंगी।
- Ubuntu layer दो बार store नहीं होगी।

3. Faster Download

- जब नई image pull करते हैं:

```
docker pull nginx
```

- तो Docker सिर्फ नई/बदली हुई layers डाउनलोड करता है।
- बाकी layers local cache से ले लेता है।

Docker Container

List Containers

Show running containers

```
docker ps
docker container ls

# List All Containers (Running + Stopped)
docker ps -a
docker container ls -a
```

Image to container/create Container

```
docker Containers run image_id/image_name
docker Containers run nginx
```

Run in background

container run in background/detached mode

- `docker run -d image_name` में `-d` का मतलब है detached mode।
- `-d` (detached mode) का मतलब है कि कंटेनर background में चलेगा।
- आप कंटेनर को स्टार्ट करेंगे, लेकिन उसका टर्मिनल या आउटपुट आपके स्क्रीन पर नहीं दिखेगा।
- कंटेनर अपने आप काम करता रहेगा और आप अपना टर्मिनल फ्री रख सकते हैं।

```
docker container run -d nginx
```

Run Interactive Mode

```
docker container run -it ubuntu bash
```

Port mapping (host → container)

- `-p` port mapping के लिए होता है, while `-d` container को detached/background mode में run करता है।

```
docker container run -p 8080:80 nginx
docker container run -d -p 8080:80 nginx
```

Run with custom name

```
docker run --name <container_name> <image_name>
docker run --name mycontainer nginx
```

Start Container

```
docker start container_id/container_name
```

Stop Container

```
docker stop container_id/container_name
```

Restart Container

```
docker restart container_id/container_name
```

docker Pause

```
docker container pause cont_id
```

docker unpause

```
docker container unpause cont_id
```

Docker inactive mode

```
docker container kill cont_id
```

Rename Container


```
# show the all conatiner
docker ps -a

# Rename container
docker container rename <old_name> <new_name>

# Container create karte waqt naam dena
docker run -d --name web_server nginx
```

Remove Container

```
# Step 1:- stop the container
docker stop <container_id_or_name>

# Step 2:- delete the container
docker rm container_id/container_name
```

- remove all containers

```
docker container rm <container_id_or_name> <container_id_or_name>
docker container prune # all container are deleted
```

- Force Remove Running Container/stop and remove the container

```
docker rm -f con_id

# Delete all containers
docker rm $(docker ps -aq)
```

container logs

```
docker container logs cont_id
docker container logs -f cont_id # tarimanl ko hold rakehga
```

Check stats

- mtb kitni memory le raha hai, kitna cpu hai

```
docker container stats --no-stream
```

Docker Exec (Enter Inside Container)

```
docker exec -it container_id bash
docker exec -it container_id sh
```

Docker Run and inspect

- mtb container ke andar jana hai

```
docker container run -d nginx:latest
# output mai container id milegi

# Now ab iske andar jana hai
docker inspect cont_id | grep -i ip
# Outpur isme ip milegi

# ab check kar
curl 172.17.0.2
# to ye hit ho jayega
```

kaunse ports par services listen kar rahi hain?

- Haan, ye Linux command hai jo mainly check karti hai ki kaunse ports par services listen kar rahi hain.

```
netstat -tulpn | grep -i LIST
```

Breakdown

- -t → TCP connections
- -u → UDP connections
- -l → Listening ports
- -p → Process/PID information
- -n → Port ko number me show karta hai, name resolve nahi karta

Work with ubuntu

```
docker container run -itd ubuntu:18.04

# create folder in local
mkdir m_folder

# create files inside folder
touch m_folder/files{1..10}.txt

# to check file go to m_folder
```

```
ls
```

```
# copy local file to container inside the temp folder
```

```
docker container cp m_folder / cont_id:/temp
```

```
# to check file in container
```

```
docker container exec -it cont_id ls -lrth /temp/m_folder
```

create image own container

```
docker container commit cont_id files:v1
```

Docker Volume?

What is a Docker Volume?

- Docker Volume is a mechanism to store data permanently outside the container.
- Normally, when a container is deleted, all data inside that container is lost. A Volume keeps the data safe even if the container stops or is removed.
- HINDI:- Docker Volume ek aisi storage jagah hai jahan container ka data permanently store hota hai.
- HINDI:- Agar aap container ke andar data save karte hain aur baad me container delete ho jata hai, to uska data bhi delete ho jata hai. Lekin agar data Volume me store hai, to container delete hone ke baad bhi data safe rehta hai.

Why do we need Volumes?

- **Persistent data storage:-** Container delete hone ke baad bhi data safe rehta hai.
- **Container Recreation:-** Agar container crash ho jaye ya naya container create karna pade, to purana data volume se wapas mil jata hai.
- **Data Sharing:-** Ek hi volume ko multiple containers use kar sakte hain.
- **Backup and Restore:-** Volumes ka backup lena aur restore karna aasaan hota hai.
- **Better Data Management:-** Application code aur data ko alag rakhne mein help karta hai.
- Data survives container deletion
- Share data between multiple containers
- Better performance than storing data inside containers

Create a Volume

```
docker volume create vol_name
```

Check Volume

```
docker volume ls
```

```
# Output:-
```

DRIVER	VOLUME NAME
local	myvolume

Container run with volume

```
# Pehle volume create karo:  
docker volume create mysql-data
```

```
# then run
docker run -d --name mysql-container -v mysql-data:/var/lib/mysql mysql
```

Alternative: Docker automatically Volume create kar sakta hai

```
docker run -d --name mycontainer -v myvolume:/app/data nginx
# Docker myvolume ko automatically create kar dega.
```

What is Data Persistence?

- Data persistence means keeping data available even after an application, server, or container is stopped, restarted, or deleted. In Docker, volumes and bind mounts are commonly used for data persistence.
- HINDI:- Data Persistence ka matlab hai ki data ko permanently save rakhna, taaki application, server ya container stop/restart/delete hone ke baad bhi data available rahe.

How do you mount a volume to a container?

- Docker Volume ko container ke saath attach/mount karne ke liye **-v** ya **--mount** option use karte hain.

1. Using -v — Simple & Common

```
docker run -d --name mycontainer \
-v myvolume:/app/data \
nginx
```

Explain

```
-v myvolume:/app/data
|           |
|           |_____ Container ke andar ka path
|_____ Docker Volume
```

Matlab myvolume ka data container ke /app/data folder mein available hoga.

2. Using --mount

- Same kaam:

```
docker run -d --name mycontainer \
--mount source=myvolume,target=/app/data \
nginx
```

Difference between Volume and Bind Mount?

- Docker khud storage location manage karta hai. | Feature | Docker Volume | Bind Mount | | -----
 -- | ----- | ----- | | Managed By | Docker | Host OS
 | | Storage Location | Docker decides location | User specifies exact path | | Easy to Use |  Yes | 
 Slightly complex | | Portability |  Better |  Less portable | | Performance |  Optimized by Docker |
 Depends on host filesystem | | Share Host Files |  Not directly |  Yes | | Common Use | Databases,
 logs, persistent data | Development, config files, source code |

Volume example with mysql

- pull mysql

```
docker image pull mysql:latest
```

- check karo

```
docker images
```

Output:-

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
mysql	latest	xxxxxxx	...	...

- Docker Volume Create karo

```
docker volume create mysql-data
```

Check

```
docker volume ls
```

DRIVER	VOLUME NAME
local	mysql-data

- MySQL Container Run karo

```
docker run -d \
  --name mysql-container \
  -e MYSQL_ROOT_PASSWORD=root123 \
  -v mysql-data:/var/lib/mysql \
  mysql
```

Output:-

```
a7f8c9d1234567890abcdef...
```

- Check Container

```
docker ps
```

```
# Output:-
```

CONTAINER ID	IMAGE	COMMAND	STATUS	PORTS	NAMES
a7f8c9d12345	mysql	"docker-entrypoint..."	Up 10 seconds	3306/tcp	mysql-container

- Volume Container mein Mount Hua ya Nahi Check karo

```
docker inspect mysql-container
```

```
"Mounts": [
  {
    "Type": "volume",
    "Name": "mysql-data",
    "Destination": "/var/lib/mysql"
  }
]
```

- MySQL ke andar Login karke Database Create karo

```
docker exec -it mysql-container mysql -uroot -p
```

```
# password
root123
```

```
# Ab MySQL prompt milega:
mysql>SHOW DATABASES;
```

```
+-----+
| Database          |
+-----+
| information_schema |
| mysql             |
| performance_schema |
| sys               |
| testdb            |
+-----+
```

docker registry?

What is docker registry?

- Docker Registry is a centralized place where Docker images are stored and from where images can be pulled or pushed using Docker commands.

- HINDI:- Docker Registry ek storage system hota hai jahan Docker images store aur manage ki jaati hain.

Docker Networking?

What is Docker Networking?

- Docker Networking allows Docker containers to communicate with:
 - Other containers
 - The Docker host machine
 - External networks (Internet)
- By default, every container gets its own isolated network namespace, IP address, and network interfaces.

Example

- Suppose you have:
 - Container A → FastAPI App
 - Container B → MySQL Database
- Docker networking allows FastAPI to connect to MySQL using the container name instead of an IP address.

```
mysql://root:password@mysql_container:3306/mydb
```

Why Docker Networking?

- Container-to-container communication
- Access applications from browser
- Connect containers to databases
- Isolate different applications

Types of Docker Networks

1. Bridge Network (Default)

- Bridge is the default Docker network that enables communication between containers running on the same host.
- When you create a container without specifying a network, Docker uses the bridge network.

```
docker run -d nginx
```

```
# Check networks:
```

```
docker network ls
```

```
# Output:-
```

NETWORK ID	NAME	DRIVER
xxxxxx	bridge	bridge

xxxxxx	host	host
xxxxxx	none	null

2. Host Network

- Container shares the host's network stack.
- No separate container IP is created.

```
docker run --network host nginx
```

- Advantage:
 - Better performance
- Disadvantage:
 - Less isolation

3. None Network

- Container has no network access.
- Used for maximum isolation.

```
docker run --network none nginx
```

4. Custom Bridge Network

- Most commonly used in real projects.
- Create network:

```
docker network create my-network
```

- Run containers:

```
docker run -d --name mysql --network my-network mysql
docker run -d --name fastapi --network my-network myapp
```

- Now FastAPI can access MySQL using:

```
mysql:3306
# without knowing the IP address.
```

Docker Compose?

What is Docker Compose?

- Docker Compose is a tool used to define and run multiple Docker containers using a single YAML file (docker-compose.yml).
- Instead of running many docker run commands manually, you can define all services in one file and start them together.

Benefits of Docker Compose

- Manage multiple containers together
- Single configuration file
- Easy startup and shutdown
- Automatic networking between services
- Ideal for development and testing

```
services:
  web:
    image: nginx
    ports:
      - "8080:80"

  db:
    image: mysql
    environment:
      MYSQL_ROOT_PASSWORD: root
```

saf

- delete the container first related to image

```
docker rmi nginx
docker rmi IMAGE_ID # Delete image by ID
docker rmi mohit2708/myfastapiapp:latest # delete with specific name
docker rmi -f IMAGE_ID # Force delete
docker image prune # Remove unused images
docker rmi $(docker images -q) # Delete all images
```

Create/build image from Dockerfile

- -t = tag
- . = Current Directory

```
docker build -t myapp .  
  
# Download a specific version  
docker build -t myapp:v1 .  
  
#build without cache  
docker build -t <image_name>:<version> . -no-cache
```

Docker all comands